SQL en Oracle Database

DML: Consultas con SELECT

© Agustín Crespo Valencia

Apuntes de Clase

Versión: Oracle 19c / 21c / 23c

Índice

1. Introducción a la Sentencia SELECT **2**
1.1. Selección Básica de Columnas . . . 2
1.2. Operadores Aritméticos . . . 2
1.3. Alias de Columnas . . . 3
1.4. El Operador de Concatenación . . . 3
1.5. Filas Duplicadas (DISTINCT) . . . 3

2. Filtrado de Datos (La cláusula WHERE) **4**
2.1. Operadores de Comparación . . . 4
2.2. Operadores Especiales de SQL . . . 6
2.2.1. BETWEEN (Rangos) . . . 6
2.2.2. IN (Listas) . . . 6
2.2.3. LIKE (Coincidencia de Patrones) . . . 6
2.2.4. IS NULL (Valores Nulos) . . . 7
2.3. Operadores Lógicos (AND, OR, NOT) . . . 7

3. Ordenación de Resultados **9**
3.1. Ordenación Básica (ASC y DESC) . . . 9
3.2. Ordenación por Múltiples Columnas . . . 9
3.3. Ordenación por Alias y Posición . . . 10
3.4. Tratamiento de Nulos en la Ordenación . . . 10
3.5. Limitación de Filas (Top-N Queries) . . . 10
3.5.1. Desglose de la Sintaxis . . . 11
3.5.2. Ejemplos Prácticos . . . 11

4. Funciones de Fila Única (Single-Row Functions) **13**
4.1. Funciones de Caracteres . . . 13
4.2. Funciones Numéricas . . . 14
4.3. Funciones de Fecha . . . 14
4.4. Funciones de Conversión . . . 15
4.5. Funciones Generales (Manejo de Nulos) . . . 15
4.6. Expresiones Condicionales (CASE y DECODE) . . . 16
4.6.1. Expresión CASE (Estándar ANSI) . . . 16
4.6.2. Función DECODE (Específica de Oracle) . . . 16

5. Funciones de Agregación y Agrupación **17**
5.1. Funciones de Grupo Más Comunes . . . 17
5.1.1. MIN y MAX . . . 17
5.1.2. SUM y AVG . . . 18
5.1.3. COUNT . . . 18
5.2. El tratamiento de NULL en Funciones de Grupo . . . 18
5.3. Creación de Grupos (Cláusula GROUP BY) . . . 19
5.3.1. Agrupación por múltiples columnas . . . 19
5.4. Filtrado de Grupos (Cláusula HAVING) . . . 19
5.5. Resumen del Orden Lógico de Ejecución . . . 21

6. Consultas Multitabla (Joins) 22
6.1. Tipos de Uniones (Joins) 22
6.2. Sintaxis ANSI vs Tradicional 22
6.3. INNER JOIN (La Intersección) 23
6.4. OUTER JOINS (Uniones Externas) 23
6.5. SELF JOIN (Auto-unión) 24
6.6. Producto Cartesiano (CROSS JOIN) 25

7. Operadores de Conjuntos (Set Operators) 26
7.1. UNION y UNION ALL 26
7.2. INTERSECT 26
7.3. MINUS 26

8. Subconsultas (Subqueries) 28
8.1. Subconsultas de Fila Única (Escalares) 28
8.2. Subconsultas de Múltiples Filas 29
8.2.1. Operador IN 29
8.2.2. Operadores ANY y ALL 29
8.3. Subconsultas Correlacionadas 30
8.3.1. Operador EXISTS 30

9. Introducción a Funciones Analíticas 32
9.1. Ranking de Filas (ROW_NUMBER, RANK, DENSE_RANK) 32
9.2. Totales Acumulados 32
9.3. Comparación con filas previas (LAG y LEAD) 33

Caso de Estudio: La Tabla EMPLEADOS

Para garantizar una comprensión profunda y práctica de las consultas SQL, utilizaremos un escenario transversal durante parte de esta guía: la gestión de recursos humanos de una corporación internacional, en concreto, de los empleados.

Trabajaremos sobre una tabla maestra denominada `empleados`. Esta tabla ha sido diseñada específicamente para contener diversas tipologías de datos (números, cadenas, fechas) y casuísticas especiales (como valores nulos en comisiones o claves foráneas), lo que nos permitirá explorar a fondo el comportamiento del motor de base de datos.

A continuación, se detalla la estructura (DDL) y el diccionario de datos:

Código 1: Definición DDL de la tabla empleados

```
CREATE TABLE empleados (
    id                  INTEGER PRIMARY KEY,
    nombre              VARCHAR2(50),
    apellido1           VARCHAR2(50)  NOT NULL,
    apellido2           VARCHAR2(50)  NOT NULL,
    email               VARCHAR2(150) NOT NULL UNIQUE,
    telefono            VARCHAR2(20),
    fecha_contratacion  DATE NOT NULL,
    puesto_id           VARCHAR2(10)  NOT NULL,
    salario             DECIMAL(8,2)  CHECK (salario > 0),
    comision_pct        DECIMAL(2,2)  CHECK (comision_pct >= 0),
    gerente_id          INTEGER,
    departamento_id     INTEGER
);
```

Análisis de columnas clave para los ejercicios:

- **salario:** Dato numérico fundamental para rangos, ordenación y cálculos.
- **comision_pct:** Porcentaje de comisión. Es vital porque contiene valores `NULL` (empleados que no cobran comisión), lo que nos servirá para estudiar el tratamiento de nulos.
- **fecha_contratacion:** Permitirá realizar filtros temporales.
- **departamento_id:** Clave foránea que se repite, ideal para agrupaciones y distinción de valores únicos.

1. Introducción a la Sentencia SELECT

La sentencia SELECT es la instrucción de manipulación de datos (DML) utilizada para recuperar información de la base de datos. En este primer apartado nos centraremos en la proyección (selección de columnas) sin aplicar filtros de filas.

SINTAXIS GENÉRICA

```
SELECT
    [DISTINCT] { * | columna1 [AS alias], columna2, expr1, ... }
FROM
    tabla;
```

1.1. Selección Básica de Columnas

La forma más elemental de consulta consiste en recuperar todas las columnas definidas en la tabla mediante el carácter comodín asterisco (*). **Personalmente, no la recomiendo.**

Código 2: Seleccionar todas las columnas

```
-- Recupera todas las filas y todas las columnas de la tabla
SELECT
    *
FROM
    empleados;
```

En entornos de producción, es recomendable especificar explícitamente las columnas deseadas (Proyección). Esto mejora el rendimiento y la claridad del código.

Código 3: Proyección de columnas específicas

```
-- Seleccionamos unicamente identificador, nombre, apellidos y correo
SELECT
    id, nombre, apellido1, apellido2, email
FROM
    empleados;
```

1.2. Operadores Aritméticos

SQL permite realizar cálculos con los datos numéricos y de fecha durante la consulta. La precedencia de operadores es la estándar matemática: multiplicación y división tienen prioridad sobre suma y resta.

Código 4: Uso de operadores aritméticos

```
-- Calculamos el salario anual (x12) y una simulacion de aumento (+300)
SELECT
    nombre, apellido1, apellido2,
    salario,
    salario * 12,
    salario + 300
FROM
    empleados;
```

1.3. Alias de Columnas

Los encabezados de las columnas resultantes suelen tomar el nombre del campo o de la expresión calculada (por ejemplo, el `salario * 12`).

Para mejorar la legibilidad, utilizamos **Alias**.

Existen dos formas de definirlos:

- Usando la palabra clave `AS` (recomendado ANSI).
- Dejando un espacio en blanco tras la columna.

Nota Importante

Si el alias contiene espacios, caracteres especiales o distingue mayúsculas de minúsculas, debe encerrarse obligatoriamente entre **comillas dobles** “ ”).

Código 5: Definición de Alias

```
-- Uso de alias para mejorar la presentacion de los encabezados
SELECT
   apellido1 AS "Primer Apellido",
   salario * 12 AS salario_anual,
   (salario + 300) * 12 AS "Salario Anual Proyectado"
FROM
   empleados;
```

1.4. El Operador de Concatenación

En Oracle, podemos unir (concatenar) columnas de texto o literales utilizando el operador de doble barra vertical (`||`).

Código 6: Concatenación de cadenas

```
-- Unimos el puesto y el apellido en una frase legible
SELECT
   puesto_id || ' asignado a ' || apellido1 AS "asignacion de puesto"
FROM
   empleados;
```

1.5. Filas Duplicadas (DISTINCT)

Por defecto, SQL muestra todas las filas resultantes, incluso si son idénticas. Para eliminar duplicados y ver valores únicos, usamos `DISTINCT` justo después de `SELECT`.

Código 7: Eliminación de duplicados

```
-- Muestra la lista de departamentos unicos que tienen empleados
-- Si hay 10 empleados en el depto 500, solo saldra una vez el 500
SELECT DISTINCT
   departamento_id
FROM
   empleados;
```

2. Filtrado de Datos (La cláusula WHERE)

Nota sobre la sensibilidad a mayúsculas y minúsculas: Oracle SQL no distingue entre mayúsculas y minúsculas (*case-insensitive*) para los nombres de tablas, columnas y palabras reservadas del lenguaje (es lo mismo escribir `SELECT` que `select`, o `EMPLEADOS` que `empleados`). Sin embargo, en esta guía utilizaremos mayúsculas para las palabras clave y minúsculas para los nombres de tablas y columnas para facilitar la lectura.

Para restringir las filas que devuelve una consulta, utilizamos la cláusula `WHERE`. Esta debe colocarse siempre después de la cláusula `FROM`.

SINTAXIS GENÉRICA

```
SELECT
   [DISTINCT] { * | columna1, ... }
FROM
   tabla
WHERE
   condición(es);
```

2.1. Operadores de Comparación

Permiten comparar el valor de una columna contra un valor específico u otra columna.

Operador	Significado
=	Igual a
>	Mayor que
>=	Mayor o igual que
<	Menor que
<=	Menor o igual que
<> o !=	Distinto de

Cuadro 1: Operadores de comparación SQL

Código 8: Filtrado por igualdad numérica

```
-- Filtramos empleados que pertenecen exactamente al departamento 500
SELECT
   nombre,
   apellido1,
   apellido2,
   departamento_id
FROM
   empleados
WHERE
   departamento_id = 500;
```

Código 9: Filtrado por cadenas de texto

```
-- NOTA: Las cadenas son Case Sensitive (distinguen mayusculas)
-- Deben ir entre comillas simples ' '
SELECT
   nombre,
   apellido1,
   apellido2,
   puesto_id
FROM
   empleados
WHERE
   apellido1 = 'Valdes';
```

Debido a que la comparación de cadenas en Oracle es **Case Sensitive**, se diferencian las mayúsculas de las minúsculas.

Un truco muy común es pasar las cadena (o los valores de las columnas) a mayúsculas (función `UPPER()`) o a minúsculas (función `LOWER()`).

A continuación nos encontramos el mismo ejemplo anterior, pero aplicando estas funciones:

Código 10: Filtrado solucionando el Case Sensitive de las cadenas

```
SELECT
   nombre,
   apellido1,
   apellido2,
   puesto_id
FROM
   empleados
WHERE
   upper(apellido1) = 'VALDES';
```

Cuando estemos comparando fechas, es fundamental tener en cuenta qué formato de fecha estamos usando en nuestra base de datos Oracle.

Si no se conoce, usaremos la función `TO_DATE()`, como en el siguiente ejemplo:

Código 11: Filtrado por fechas

```
-- Oracle utiliza un formato de fecha por defecto (usualmente DD/MM/RR)
-- Es recomendable usar TO_DATE para formatear la fecha:
SELECT
   nombre,
   apellido1,
   apellido2,
   fecha_contratacion
FROM
   empleados
WHERE
   fecha_contratacion = TO_DATE('25/10/03', 'RR-MM-DD');
```

De esta forma, quitamos la ambigüedad de si la fecha `'25/10/03'` es el 3 de octubre del 2025, o el 25 de octubre del 2003.

2.2. Operadores Especiales de SQL

Además de los matemáticos, SQL dispone de operadores potentes para rangos, listas y patrones.

2.2.1. BETWEEN (Rangos)

Selecciona valores dentro de un intervalo (incluye los límites).

Código 12: Uso de BETWEEN

```
-- Empleados con salario entre 2500 y 3500 (ambos inclusive)
SELECT nombre, apellido1, apellido2, salario
FROM
    empleados
WHERE
    salario BETWEEN 2500 AND 3500;
```

2.2.2. IN (Listas)

Verifica si un valor coincide con algún elemento de una lista específica.

Código 13: Uso de IN

```
-- Empleados Gerentes (SA_MAN) o Representantes (SA_REP) de ventas
SELECT nombre, apellido1, apellido2, puesto_id
FROM
    empleados
WHERE
    puesto_id IN ('SA_MAN', 'SA_REP');
```

2.2.3. LIKE (Coincidencia de Patrones)

Se utiliza para buscar cadenas que cumplan un patrón, usando comodines:

- % : Representa cero o muchos caracteres.
- _ : Representa exactamente un carácter.

Código 14: Uso de LIKE con comodines

```
-- Apellido que empiezan por 'S' (S seguido de cualquier cosa)
SELECT nombre, apellido1, apellido2
FROM
    empleados
WHERE
    apellido1 LIKE 'S%';
```

Código 15: Uso de LIKE específico

```
-- Nombres que tienen una 'o' como segunda letra
-- El guion bajo salta la primera letra
SELECT nombre, apellido1, apellido2
FROM
    empleados
WHERE
    nombre LIKE '_o%';
```

2.2.4. IS NULL (Valores Nulos)

Un error común es intentar comparar nulos con `= NULL`. Esto siempre devuelve falso o desconocido. Se debe usar obligatoriamente `IS NULL` o `IS NOT NULL`.

Código 16: Filtrado de nulos

```
-- Buscamos empleados que NO tienen gerente asignado (el jefe supremo)
SELECT
   nombre,
   apellido1,
   gerente_id
FROM
   empleados
WHERE
   gerente_id IS NULL;
```

2.3. Operadores Lógicos (AND, OR, NOT)

Permiten combinar múltiples condiciones en la cláusula WHERE.

- **AND**: Ambas condiciones deben ser verdaderas.
- **OR**: Al menos una condición debe ser verdadera.
- **NOT**: Invierte el resultado de la condición.

Código 17: Uso de AND

```
-- Empleados del depto 100 Y que ganan mas de 20000
SELECT
   nombre,
   apellido1,
   apellido2,
   departamento_id,
   salario
FROM
   empleados
WHERE
   departamento_id = 100
      AND salario > 20000;
```

Código 18: Precedencia de Operadores sin paréntesis

```
-- El AND tiene prioridad sobre el OR.
-- Aqui: (Dpto 100 y salario >= 10000) O (Dpto 500 y cualquier salario)
SELECT
   nombre,
   apellido1,
   apellido2,
   departamento_id,
   salario
FROM
   empleados
WHERE
   departamento_id = 500
      OR departamento_id = 100
      AND salario >= 10000;
```

Debido a que las prioridades de los condicionales pueden ser confusas (igualmente aplica cuando se realizan operaciones matemáticas), se recomienda agregar paréntesis para dar preferencia a una operación sobre las demás.

Código 19: Uso de Paréntesis

```
-- Para forzar la prioridad del OR, usamos parentesis
-- (Depto 100 o 200) Y ADEMAS ganar mas de 10000
SELECT
   nombre,
   apellido1,
   apellido2,
   departamento_id,
   salario
FROM
   empleados
WHERE
   (departamento_id = 100 OR departamento_id = 200)
      AND salario > 10000;
```

3. Ordenación de Resultados

En el modelo relacional, el orden de las filas en una tabla no está garantizado.

Si realizamos una consulta SELECT sin especificar un orden, Oracle devolverá los datos en el orden en que los encuentre más eficiente recuperar.

SINTAXIS GENÉRICA

```
SELECT
    [DISTINCT] { * | columna1 [AS alias], columna2, expr1, expr2 ... }
FROM
    tabla
WHERE
    condición(es)
ORDER BY
    {columna | alias | posición} [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ];
```

Para garantizar una secuencia específica, utilizamos la cláusula ORDER BY. Esta debe ser siempre la **última cláusula** de la sentencia SQL estándar.

3.1. Ordenación Básica (ASC y DESC)

Por defecto, la ordenación es ascendente (ASC): de menor a mayor en números, de la A a la Z en texto, y de lo más antiguo a lo más reciente en fechas.

Para invertir el orden, se debe especificar DESC.

Código 20: Ordenación por fecha de contratación

```
-- Listar empleados desde el más reciente al más antiguo
SELECT
   nombre,
   apellido1,
   fecha_contratacion
FROM
   empleados
ORDER BY
   fecha_contratacion DESC;
```

3.2. Ordenación por Múltiples Columnas

Es posible ordenar por más de una columna, como en este ejemplo:

Código 21: Ordenación múltiple

```
-- Ordenar por departamento (ascendente) y, dentro de cada uno,
-- por salario de mayor a menor
SELECT
   departamento_id,
   apellido1,
   salario
FROM
   empleados
ORDER BY
   departamento_id ASC,
   salario DESC;
```

El orden de ejecución es secuencial: primero ordena por la primera columna especificada (en este caso: `departamento_id`; si hay valores idénticos (empates), ordena esas filas por la segunda columna del `ORDER BY` (en este caso: `salario`), y así sucesivamente.

3.3. Ordenación por Alias y Posición

Oracle permite ordenar utilizando el alias definido en la cláusula `SELECT`, lo cual es muy útil cuando ordenamos por expresiones calculadas.

También permite usar la posición numérica de la columna (aunque esto último se desaconseja por falta de legibilidad).

Código 22: Uso de Alias en ORDER BY

```
-- Calculamos el salario anual y ordenamos por ese resultado
SELECT
   nombre,
   salario * 12 AS salario_anual
FROM
   empleados
ORDER BY
   salario_anual DESC;
```

3.4. Tratamiento de Nulos en la Ordenación

Los valores `NULL` presentan un comportamiento particular al ordenar:

- En orden `ASC`, los nulos aparecen al **final**.
- En orden `DESC`, los nulos aparecen al **principio**.

Podemos alterar este comportamiento usando `NULLS FIRST` o `NULLS LAST`.

Código 23: Control de Nulos

```
-- Ordenar por comisiones. Queremos ver primero los que tienen comisión
-- más baja, pero los que no tienen (NULL) queremos que salgan al final,
-- no al principio (que sería el defecto en ASC)
SELECT
   nombre,
   apellido1,
   comision_pct
FROM
   empleados
ORDER BY
   comision_pct ASC NULLS LAST;
```

3.5. Limitación de Filas (Top-N Queries)

A partir de Oracle 12c, se introdujo una sintaxis estándar ANSI para limitar el número de filas retornadas. Anteriormente, esto requiera el uso complejo de subconsultas y la pseudocolumna `ROWNUM`.

Esta sintaxis es ideal para responder preguntas de negocio como *"¿Quiénes son los 5 mejores vendedores?"* o para *implementar la paginación en aplicaciones web*.

SINTAXIS GENÉRICA

```
SELECT ...
FROM ...
WHERE ...
ORDER BY ...
OFFSET n ROWS
FETCH { FIRST | NEXT } m [ PERCENT ] ROWS { ONLY | WITH TIES };
```

3.5.1. Desglose de la Sintaxis

La cláusula se compone de dos partes opcionales:

- `OFFSET n ROWS`**:** Indica cuántas filas se deben **saltar** antes de empezar a devolver resultados. Si se omite, se empieza desde la primera fila (fila 0).
- `FETCH ... ROWS`**:** Indica cuántas filas se deben **devolver** tras el salto.

Dentro de `FETCH`, disponemos de varias opciones potentes:

`FIRST` **o** `NEXT`**:** Son sinónimos. Se usa uno u otro por legibilidad gramatical.
Como, p. ej.: *"Fetch first 5..."* o *"Offset 5 ... fetch next 5"*.

`PERCENT`**:** En lugar de un número fijo de filas, calcula el porcentaje del total de resultados.

`ONLY`**:** Devuelve el número exacto de filas solicitado, cortando drásticamente el resultado.

`WITH TIES`**:** Si la última fila retornada tiene valores idénticos en las columnas del `ORDER BY` con las filas siguientes, estas filas adicionales también se devuelven (se incluyen los empates).

3.5.2. Ejemplos Prácticos

Caso 1: Los N mejores (Limitación estricta)
El caso más común. Queremos exactamente 5 filas, ni una más.

Código 24: Obtener los 5 salarios más altos (ONLY)

```
-- Seleccionamos estrictamente 5 empleados.
-- Si hay un empate en la posicion 5, Oracle elegira arbitrariamente
-- cual mostrar y cual cortar.
SELECT
   nombre,
   salario
FROM
   empleados
ORDER BY
   salario DESC
FETCH FIRST 5 ROWS ONLY;
```

Caso 2: Gestión de Empates (WITH TIES)

Imaginemos que en la posición 5 hay un empleado que cobra 10.000, pero en la posición 6 hay otro que **también** cobra 10.000. Con `ONLY`, el segundo se quedaría fuera injustamente. Con `WITH TIES`, ambos aparecen.

Código 25: Inclusión de empates

```
-- Recupera los 5 primeros, pero si el 5to empata con el 6to
-- (y sucesivos), tambien los muestra. Puede devolver mas de 5 filas.
SELECT
   nombre,
   salario
FROM
   empleados
ORDER BY
   salario DESC
FETCH FIRST 5 ROWS WITH TIES;
```

Caso 3: Top por Porcentaje

Útil para análisis estadísticos, como obtener el decil superior.

Código 26: Obtener el 20 % superior de la plantilla

```
-- Si la empresa tiene 100 empleados, devolvera los 20 con mayor sueldo
SELECT
   nombre,
   salario
FROM
   empleados
ORDER BY
   salario DESC
FETCH FIRST 20 PERCENT ROWS ONLY;
```

Caso 4: Paginación de Resultados

Es la combinación de `OFFSET` (saltar páginas anteriores) y `FETCH` (mostrar la página actual).

Código 27: Paginación (Página 2)

```
-- Escenario: Mostramos 5 resultados por pagina.
-- Para ver la pagina 2, saltamos los 5 primeros (OFFSET)
-- y cogemos los 5 siguientes (FETCH).
SELECT
   id,
   nombre,
   apellido1,
   apellido2
FROM
   empleados
ORDER BY
   apellido1, apellido2
OFFSET 5 ROWS FETCH NEXT 5 ROWS ONLY;
```

4. Funciones de Fila Única (Single-Row Functions)

Las funciones de fila única operan sobre cada fila individualmente y devuelven un resultado por fila. Pueden modificar tipos de datos, realizar cálculos o formatear la salida.

Se clasifican principalmente en: Caracteres, Numéricas, Fechas, Conversión y Generales.

4.1. Funciones de Caracteres

Estas sólo son algunas de las funciones más comunes para trabajo sobre cadenas, pero hay muchas más (que se irán aprendiendo con el uso):

Función	Propósito
`LENGTH(cadena)`	Devuelve la longitud de la cadena.
`LOWER(cadena)`	Convierte a minúsculas.
`UPPER(cadena)`	Convierte a mayúsculas.
`INITCAP(cadena)`	Convierte la primera letra de cada palabra a mayúscula.
`SUBSTR(cad, m, n)`	Extrae una subcadena desde la posición m, max. hasta n caracteres.
`CONCAT(cad1, cad2)`	Concatena dos cadenas de texto (equivalente a ||).
`INSTR(cad, sub)`	Devuelve la posición numérica donde aparece la subcadena.
`TRIM(cadena)`	Elimina espacios (o caracteres) de los extremos.
`REPLACE(cad, a, b)`	Reemplaza las ocurrencias de las cadenas 'a' por 'b'.
`LPAD(cad, n, 'str')`	Rellena a la izquierda con 'str' hasta alcanzar longitud n.
`RPAD(cad, n, 'str')`	Rellena a la derecha con 'str' hasta alcanzar longitud n.

Cuadro 2: Principales funciones de caracteres

Código 28: Manipulación de mayúsculas y minúsculas

```
-- Estandarizar el formato de nombres para un reporte
SELECT
   UPPER(apellido1) AS apellido1_mayus,
   INITCAP(nombre) AS nombre_capitalizado
FROM
   empleados;
```

Código 29: Manipulación de cadenas

```
-- Extraer las 3 primeras letras del nombre
-- Calcular la longitud del primer apellido
SELECT
   nombre,
   SUBSTR(nombre, 1, 3) AS abreviatura,
   LENGTH(apellido1) AS longitud_apellido1
FROM
   empleados;
```

4.2. Funciones Numéricas

Permiten redondear, truncar o realizar operaciones modulares sobre números.

Código 30: ROUND vs TRUNC

```
-- ROUND redondea al entero o decimal más cercano
-- TRUNC simplemente "corta" el número sin redondear
SELECT
   salario / 3 AS division_exacta,
   ROUND(salario / 3, 2) AS redondeo_2_decimales,
   TRUNC(salario / 3, 2) AS truncado_2_decimales
FROM
   empleados
WHERE
   id = 100;
```

4.3. Funciones de Fecha

Oracle almacena las fechas internamente como números, lo que permite realizar aritmética con ellas (sumar días a una fecha, restar dos fechas para obtener días de diferencia, etc.).

Función	Descripción
SYSDATE	Devuelve la fecha y hora actual del servidor.
MONTHS_BETWEEN	Devuelve el número de meses entre dos fechas.
ADD_MONTHS	Suma un número de meses a una fecha.
LAST_DAY	Devuelve el último día del mes de la fecha dada.
NEXT_DAY	Devuelve la fecha del siguiente día de la semana indicado.

Cuadro 3: Funciones de manipulación de fechas

Código 31: Cálculos con fechas

```
-- Antigüedad en semanas y fecha de revisión salarial (6 meses después)
SELECT
   id,
   (SYSDATE - fecha_contratacion) / 7 AS semanas_trabajadas,
   ADD_MONTHS(fecha_contratacion, 6) AS revision_semestral
FROM
   empleados;
```

4.4. Funciones de Conversión

En ocasiones es necesario convertir explícitamente un dato de un tipo a otro (por ejemplo, mostrar una fecha con un formato de texto específico o convertir una cadena '123' a número).

SINTAXIS GENÉRICA

```
TO_CHAR(numero | fecha [, 'formato'])
TO_DATE(cadena [, 'formato'])
TO_NUMBER(cadena [, 'formato'])
```

Los modelos de formato son muy potentes. Algunos ejemplos comunes:

- **Fechas:** YYYY (año 4 dígitos), MM (mes numérico), MONTH (nombre mes), DD (día), DAY (nombre día).
- **Números:** 9 (dígito), 0 (dígito forzando ceros), G (separador miles), D (separador decimal), L (símbolo moneda local).

Código 32: Uso de TO_CHAR con Fechas y Números

```
-- Formatear fecha a texto completo y salario a formato moneda
SELECT
   nombre,
   TO_CHAR(fecha_contratacion, 'DD "de" Month "del" YYYY') AS
      fecha_texto,
   TO_CHAR(salario, 'L99G999D99') AS salario_formato
FROM
   empleados;
```

4.5. Funciones Generales (Manejo de Nulos)

Estas funciones son vitales para evitar errores aritméticos o lógicos cuando los datos contienen NULL.

- NVL(expr1, expr2): Si expr1 es nulo, devuelve expr2. Si no, devuelve expr1.
- NVL2(expr1, expr2, expr3): Si expr1 NO es nulo, devuelve expr2. Si ES nulo, devuelve expr3.
- COALESCE(expr1, expr2, ..., exprn): Devuelve la primera expresión no nula de la lista.

Código 33: Uso de NVL para cálculos

```
-- Si comision_pct es NULL, la multiplicación daría NULL.
-- Usamos NVL para convertir el NULL en 0 temporalmente.
SELECT
   nombre || ' ' || apellido1 || ' ' || apellido2 AS nombre_completo,
   salario,
   comision_pct,
   (salario * 12) + (salario * 12 * NVL(comision_pct, 0)) AS
      ingreso_total
FROM
   empleados;
```

4.6. Expresiones Condicionales (CASE y DECODE)

Permiten implementar lógica `IF-THEN-ELSE` dentro de una consulta SQL.

4.6.1. Expresión CASE (Estándar ANSI)

Es la forma recomendada por ser estándar y más legible.

Código 34: Clasificación con CASE

```
-- Categorizar empleados según su salario
SELECT
   nombre || ' ' || apellido1 || ' ' || apellido2 AS nombre_completo,
   salario,
   CASE
      WHEN salario < 3000 THEN 'Junior'
      WHEN salario BETWEEN 3000 AND 8000 THEN 'Especialista'
      WHEN salario > 8000 THEN 'Directivo'
      ELSE 'No clasificado'
   END AS categoria_salarial
FROM
   empleados;
```

4.6.2. Función DECODE (Específica de Oracle)

Su sintaxis es más compacta pero menos intuitiva. Funciona por pares *valor-búsqueda, resultado*.

Código 35: Uso de DECODE

```
-- Simular un aumento según el puesto
-- IT_PROG sube 10%, ST_CLERK sube 5%, el resto 0%
SELECT
   nombre || ' ' || apellido1 || ' ' || apellido2 AS nombre_completo,
   puesto_id,
   salario,
   DECODE(puesto_id,
          'IT_PROG',  salario * 1.10,
          'ST_CLERK', salario * 1.05,
          salario) AS nuevo_salario
FROM
   empleados;
```

5. Funciones de Agregación y Agrupación

A diferencia de las funciones de fila única (vistas en el apartado anterior), las funciones de agregación (o de grupo) operan sobre un conjunto de filas para devolver **un único resultado** por cada grupo.

Son fundamentales para la generación de informes, ya que permiten responder preguntas como: "¿Cuál es el salario promedio?", "¿Cuántos empleados hay en cada departamento?" o "¿Cuál es el gasto total en nóminas?".

La sintaxis básica es la siguiente

SINTAXIS GENÉRICA

```
SELECT
    funcion_agregacion(columna)
FROM
    tabla
[WHERE condición]
[GROUP BY columna]
[HAVING condición_grupo];
```

5.1. Funciones de Grupo Más Comunes

Oracle ofrece numerosas funciones de agregación, pero las cinco más utilizadas son:

Función	Descripción	Tipos de Datos
AVG()	Calcula el promedio (media aritmética).	Solo Numérico
SUM()	Calcula la suma total.	Solo Numérico
MAX()	Encuentra el valor máximo.	Cualquiera
MIN()	Encuentra el valor mínimo.	Cualquiera
COUNT()	Cuenta el número de filas o valores.	Cualquiera

Cuadro 4: Funciones de agregación principales

5.1.1. MIN y MAX

Se pueden utilizar para datos numéricos, fechas y cadenas de texto.

Código 36: Uso de MIN y MAX

```
-- Obtenemos el salario mas alto, el mas bajo y la fecha de la
-- contratacion mas antigua de toda la empresa
SELECT
   MAX(salario) AS salario_maximo,
   MIN(salario) AS salario_minimo,
   MIN(fecha_contratacion) AS empleado_mas_antiguo
FROM
   empleados;
```

5.1.2. SUM y AVG

Exclusivas para datos numéricos.

Código 37: Uso de SUM y AVG

```
-- Gasto total en salarios y salario promedio de la empresa
SELECT
   SUM(salario) AS nomina_total,
   ROUND(AVG(salario), 2) AS salario_promedio
FROM
   empleados;
```

5.1.3. COUNT

Esta función tiene tres variantes importantes que suelen confundirse:

- `COUNT(*)`: Cuenta el número total de filas, incluyendo duplicados y filas con valores nulos.
- `COUNT(columna)`: Cuenta el número de filas donde `columna` **NO** es nulo.
- `COUNT(DISTINCT columna)`: Cuenta el número de valores únicos no nulos.

Código 38: Diferentes usos de COUNT

```
-- 1o. Total de empleados (filas en la tabla)
-- 2o. Empleados que tienen comision (ignora los nulos en comision_pct)
-- 3o. Numero de departamentos distintos que tienen empleados
SELECT
   COUNT(*) AS total_empleados,
   COUNT(comision_pct) AS empleados_con_comision,
   COUNT(DISTINCT departamento_id) AS deptos_activos
FROM
   empleados;
```

5.2. El tratamiento de NULL en Funciones de Grupo

Una regla de oro en SQL es que **todas las funciones de grupo ignoran los valores NULL**, excepto `COUNT(*)`.

Esto es crítico al calcular promedios. Si tenemos 3 empleados con comisiones: 100, 200 y NULL:

- `AVG(comision)` calcula $(100 + 200)/2 = 150$. (Ignora al tercero).
- Si queremos considerar al tercero como 0, debemos usar la función `NVL`.

Código 39: AVG con y sin NVL

```
-- Promedio 1: Solo considera a los que tienen comision
-- Promedio 2: Considera a todos (los nulos cuentan como 0)
SELECT
   AVG(comision_pct) AS promedio_real,
   AVG(NVL(comision_pct, 0)) AS promedio_ajustado
FROM
   empleados;
```

5.3. Creación de Grupos (Cláusula GROUP BY)

Hasta ahora, hemos tratado a toda la tabla como un solo grupo. Sin embargo, lo habitual es querer estadísticas divididas por categorías (por ejemplo, "salario medio *por departamento*").

Para ello utilizamos la cláusula `GROUP BY`.

Regla Fundamental del GROUP BY

Cualquier columna que aparezca en el `SELECT` y que **NO** esté dentro de una función de agregación, **DEBE** aparecer obligatoriamente en la cláusula `GROUP BY`.

Código 40: Agrupación básica por una columna

```
-- Salario promedio por cada departamento
SELECT
   departamento_id,
   ROUND(AVG(salario), 2) AS salario_medio
FROM
   empleados
GROUP BY
   departamento_id;
```

5.3.1. Agrupación por múltiples columnas

Podemos desglosar los grupos en subgrupos listando varias columnas en el `GROUP BY`.

Código 41: Agrupación múltiple

```
-- Contar empleados por departamento y puesto
-- (Cuantos -por tipo- hay en el depto 100, en el 200, ...)
SELECT
   departamento_id,
   puesto_id,
   COUNT(*) AS cantidad_empleados
FROM
   empleados
GROUP BY
   departamento_id,
   puesto_id
ORDER BY
   departamento_id;
```

5.4. Filtrado de Grupos (Cláusula HAVING)

Un error muy común es intentar filtrar resultados agregados usando `WHERE`.
El `WHERE` **filtra filas** antes de que se formen los grupos.
El `HAVING` **filtra grupos** después de que se han calculado las agregaciones.

SINTAXIS GENÉRICA

```
SELECT columna, funcion_agregacion
FROM tabla
WHERE condición(es)
GROUP BY
    columna
HAVING
    funcion_agregacion condicion;
```

Código 42: Error común: Usar WHERE para agregados

```
-- INCORRECTO: Esto generara un error ORA-00934
-- No se puede usar AVG en el WHERE
SELECT
    departamento_id, AVG(salario)
FROM
    empleados
WHERE
    AVG(salario) > 8000
GROUP BY
    departamento_id;
```

Código 43: Correcto: Uso de HAVING

```
-- Seleccionamos los departamentos cuyo salario MAXIMO supere los 10000
SELECT
   departamento_id,
   MAX(salario)
FROM
   empleados
GROUP BY
   departamento_id
HAVING
   MAX(salario) > 10000;
```

Podemos combinar ambas cláusulas: usar `WHERE` para restringir qué filas entran en el cálculo y `HAVING` para filtrar el resultado final del grupo.

Código 44: Combinación de WHERE y HAVING

```
-- WHERE: Excluimos a los gerentes (puesto no tiene 'MAN' ni 'MGR')
-- GROUP BY: Agrupamos por departamento
-- HAVING: Mostramos solo deptos con mas de 5 empleados restantes
SELECT
   departamento_id,
   COUNT(*)
FROM
   empleados
WHERE
   puesto_id NOT LIKE '%MAN'
   AND puesto_id NOT LIKE '%MGR'
GROUP BY
   departamento_id
HAVING
   COUNT(*) > 5;
```

5.5. Resumen del Orden Lógico de Ejecución

Para escribir consultas complejas sin errores, es vital entender el orden en el que Oracle procesa la sentencia, que es distinto al orden de escritura:

1. **FROM**: Identifica la tabla.
2. **WHERE**: Filtra las filas (no admiten agregados).
3. **GROUP BY**: Agrupa las filas resultantes.
4. **HAVING**: Filtra los grupos (admite agregados).
5. **SELECT**: Selecciona las columnas y ejecuta funciones.
6. **ORDER BY**: Ordena el resultado final.

6. Consultas Multitabla (Joins)

Hasta ahora, todos nuestros ejemplos extraían datos de (`empleados`), es decir, siempre hemos trabajado con una única tabla.

Sin embargo, la potencia de las bases de datos relacionales reside en su capacidad para relacionar información dispersa en múltiples tablas.

Para realizar los ejemplos de esta sección, introduciremos conceptualmente una segunda tabla llamada `departamentos`, vinculada a la tabla de empleados a través de la columna común `departamento_id`.

Código 45: Estructura básica de la tabla DEPARTAMENTOS

```
CREATE TABLE departamentos (
   id            INTEGER PRIMARY KEY,
   nombre        VARCHAR2(30) NOT NULL,
   ubicacion_id  INTEGER
);
```

La columna `id` es la **Primary Key** (PK) en la tabla `departamentos` y actúa como **Foreign Key** (FK) en la tabla `empleados`, en la columna denominada `departamento_id`. Este vínculo es lo que nos permite cruzar los datos.

6.1. Tipos de Uniones (Joins)

Existen varios tipos de uniones según los datos que queramos recuperar:

- **INNER JOIN:** Devuelve solo las filas donde hay coincidencia en ambas tablas.
- **LEFT OUTER JOIN:** Devuelve todas las filas de la tabla izquierda, coincidan o no, y las de la tabla de la derecha que coincidan.
- **RIGHT OUTER JOIN:** Devuelve todas las filas de la tabla derecha, y de la tabla de la izquierda sólo las que coincidad.
- **FULL OUTER JOIN:** Devuelve todo el contenido de ambas tablas, cruzando lo que coincida y rellenando con NULL lo que no.
- **CROSS JOIN:** Producto cartesiano (combinación de todos con todos).

6.2. Sintaxis ANSI vs Tradicional

Oracle soporta dos sintaxis. La *"tradicional"* (usando la cláusula `WHERE` para unir) y la sintaxis estándar **ANSI SQL:1999** (usando la palabra clave `JOIN`).

Recomendación

En esta guía utilizaremos exclusivamente la sintaxis **ANSI (JOIN ... ON)**, ya que es más clara, separa la lógica de unión de la de filtrado, y es el estándar de la industria.

6.3. INNER JOIN (La Intersección)

Es el tipo de join más común. Recupera únicamente las filas que tienen valores coincidentes en la columna de unión de ambas tablas. Si un empleado tiene un `departamento_id` que no existe en la tabla departamentos (o es `NULL`), ese empleado no aparecerá.

SINTAXIS GENÉRICA

```
SELECT
    t1.columna, t2.columna
FROM
    tabla1 t1
    [INNER] JOIN tabla2 t2 ON t1.columna_clave = t2.columna_clave;
```

Es altamente recomendable usar **Alias de Tabla** (letras cortas como ‘e‘ o ‘d‘) para prefijar las columnas y evitar errores de ambigüedad (cuando dos tablas tienen una columna con el mismo nombre).

Código 46: INNER JOIN básico

```
-- Queremos ver el nombre del empleado y el nombre de su departamento.
-- El nombre del empleado esta en 'e', el del departamento en 'd'.
SELECT
   e.nombre AS "Nombre",
   e.apellido1 || ' ' || e.apellido2 AS "Apellidos",
   d.nombre AS "Departamento"
FROM
   empleados e INNER JOIN departamentos d
      ON e.departamento_id = d.id;
```

Podemos unir más de dos tablas secuencialmente. Por ejemplo, si tuviéramos una tabla `ubicaciones`:

Código 47: Unión de tres tablas

```
-- Empleado, su departamento y la ciudad donde esta
SELECT
   e.nombre AS "Nombre",
   e.apellido1 || ' ' || e.apellido2 AS "Apellidos",
   d.nombre,
   u.ciudad
FROM
   empleados e JOIN departamentos d
      ON e.departamento_id = d.id
   JOIN ubicaciones u
      ON d.ubicacion_id = u.id;
```

6.4. OUTER JOINS (Uniones Externas)

A veces necesitamos listar filas de una tabla aunque no tengan correspondencia en la otra. Por ejemplo: *“Listar todos los empleados, tengan o no departamento asignado”* o *“Listar todos los departamentos, aunque no tengan empleados”*.

LEFT OUTER JOIN

Toma todos los registros de la tabla de la **IZQUIERDA** (la primera mencionada en el `FROM`) y busca coincidencias en la derecha. Si no encuentra, rellena con `NULL`.

Código 48: LEFT JOIN

```
-- Listar TODOS los empleados y sus departamentos.
-- Si hay un empleado nuevo sin departamento (NULL), aparecera tambien.
-- (A la derecha, nombre saldra vacio/NULL).
SELECT
   e.nombre,
   e.apellido1,
   d.nombre
FROM
   empleados e LEFT JOIN departamentos d
      ON e.departamento_id = d.id;
```

RIGHT OUTER JOIN

Es el inverso del anterior. Prioriza la tabla de la **DERECHA**.

Código 49: RIGHT JOIN

```
-- Listar TODOS los departamentos y los empleados que hay en ellos.
-- Si un departamento esta vacio (sin empleados), aparecera en la lista.
-- (A la izquierda, los datos del empleado saldran NULL).
SELECT
   e.nombre,
   e.apellido1,
   e.apellido2
   d.nombre
FROM
   empleados e RIGHT JOIN departamentos d
      ON e.departamento_id = d.id;
```

FULL OUTER JOIN

Es una combinación de ambos. Muestra todo lo de la izquierda y todo lo de la derecha. Si coinciden se unen, si no, se muestran con NULL en el lado contrario.

Código 50: FULL JOIN

```
-- Mostrar todo: Empleados sin depto y Deptos sin empleados
SELECT
   e.nombre || ' ' || e.apellido1 AS "empleado",
   d.nombre AS "departamento"
FROM
   empleados e FULL JOIN departamentos d
      ON e.departamento_id = d.id;
```

6.5. SELF JOIN (Auto-unión)

En ocasiones, una tabla contiene una clave foránea que referencia a la propia tabla (relación recursiva). El ejemplo clásico en nuestra tabla `empleados` es la columna `gerente_id`, que contiene el `id` de otro empleado (su jefe).

Para resolver esto, imaginamos que tenemos dos copias de la misma tabla: una la llamamos "Trabajador" la otra "Jefe".

Código 51: Self Join: Trabajador y Jefe

```
-- Queremos una lista que diga: "El trabajador X trabaja para el jefe Y"
-- 't' representa la fila del trabajador
-- 'j' representa la fila del jefe
SELECT
   t.nombre || ' ' || t.apellido1 AS trabajador,
   j.nombre || ' ' || j.apellido1 AS jefe
FROM
   empleados t JOIN empleados j
      ON t.gerente_id = j.id;
```

Nota sobre Self Joins

Si usamos un `INNER JOIN` en el ejemplo anterior, el "Jefe Supremo" (que no tiene jefe, su `gerente_id` es NULL) desaparecería de los resultados. Si queremos incluirlo indicando que no tiene jefe, deberíamos usar un `LEFT JOIN`.

6.6. Producto Cartesiano (CROSS JOIN)

Ocurre cuando unimos dos tablas sin especificar ninguna condición de unión (o usando explícitamente `CROSS JOIN`). El resultado es la combinación de **todas** las filas de la primera tabla con **todas** las de la segunda.

Si la tabla A tiene 100 filas y la tabla B tiene 100 filas, el resultado tendrá $100 \times 100 = 10{,}000$ filas. Generalmente, esto es un error lógico o de rendimiento, salvo en casos muy específicos de generación de datos.

Código 52: CROSS JOIN (Peligroso)

```
-- Genera todas las combinaciones posibles de empleados y departamentos
SELECT
   e.nombre,
   e.apellido1,
   e.apellido2,
   d.nombre
FROM
   empleados e
   CROSS JOIN departamentos d;
```

7. Operadores de Conjuntos (Set Operators)

Mientras que los *Joins* combinan tablas horizontalmente (añadiendo columnas de otra tabla a la misma fila), los operadores de conjuntos combinan resultados verticalmente (añadiendo filas de una segunda consulta debajo de las de la primera).

Para utilizar estos operadores, las consultas deben cumplir dos reglas estrictas:

1. Deben tener el **mismo número de columnas**.
2. Las columnas deben ser de **tipos de datos compatibles** y en el mismo orden.

7.1. UNION y UNION ALL

Se utilizan para sumar los resultados de dos consultas.

Operador	Comportamiento
`UNION`	Combina los resultados y **elimina duplicados**. Ordena el resultado (más lento).
`UNION ALL`	Combina los resultados y **mantiene duplicados**. No ordena (más rápido).

Cuadro 5: Diferencia entre UNION y UNION ALL

Código 53: Ejemplo de UNION

```
-- Lista combinada de puestos actuales y puestos historicos
-- (Suponiendo que existe una tabla job_history)
SELECT puesto_id FROM empleados
UNION
SELECT puesto_id FROM job_history;
```

Si un empleado tuvo el puesto 'ST_CLERK' en el pasado y lo tiene ahora, `UNION` mostrará 'ST_CLERK' una sola vez. `UNION ALL` lo mostraría dos veces.

7.2. INTERSECT

Devuelve únicamente las filas que son comunes a ambas consultas (la intersección matemática).

Código 54: Uso de INTERSECT

```
-- Puestos que existen en la tabla de empleados Y TAMBIEN
-- en la tabla de historial (alguien tuvo ese puesto y alguien lo tiene)
SELECT puesto_id FROM empleados
INTERSECT
SELECT puesto_id FROM job_history;
```

7.3. MINUS

Devuelve las filas de la primera consulta que **NO** están presentes en la segunda consulta (la resta matemática $A - B$).

Código 55: Uso de MINUS

```
-- Empleados que NO pertenecen a ningun departamento registrado
-- en la tabla de departamentos (integridad referencial rota o nuevos)
SELECT departamento_id FROM empleados
MINUS
SELECT departamento_id FROM departamentos;
```

También es muy útil para responder preguntas tipo: "¿Qué departamentos no tienen ningún empleado?".

Código 56: Departamentos vacíos usando MINUS

```
-- Todos los departamentos MENOS los departamentos que tienen gente
SELECT departamento_id FROM departamentos
MINUS
SELECT departamento_id FROM empleados;
```

8. Subconsultas (Subqueries)

Una subconsulta es una sentencia `SELECT` anidada dentro de otra sentencia SQL. Se utilizan cuando necesitamos realizar una consulta cuyos criterios de búsqueda dependen de los resultados de otra consulta previa.

Por ejemplo: *¿Quién cobra más que el salario promedio?*

Para responder a esto, primero necesitamos saber cuál es el salario promedio (consulta interna) y luego filtrar a los empleados (consulta externa).

SINTAXIS GENÉRICA

```
SELECT lista_select
    FROM tabla_A
    WHERE expr operador (SELECT lista_select FROM tabla_B);
```

Reglas Básicas

- La subconsulta debe ir encerrada entre paréntesis.
- Se suele colocar en el lado derecho de la condición de comparación.
- No suelen llevar cláusula `ORDER BY` (salvo para Top-N queries).

8.1. Subconsultas de Fila Única (Escalares)

La subconsulta devuelve **una sola fila y una sola columna** a la consulta externa. Se utilizan con operadores de comparación de fila única: =, >, >=, <, <=, <>.

Código 57: Filtrar basándose en un cálculo agregado

```
-- Empleados que ganan más que el promedio de toda la empresa
SELECT
   nombre || ' ' || apellido1 || ' ' || apellido2 AS empleado,
   salario
FROM
   empleados
WHERE
   salario > (SELECT AVG(salario) FROM empleados);
```

Código 58: Filtrar basándose en otro registro

```
-- Empleados que tienen el mismo puesto que 'Valdes'
SELECT
   nombre || ' ' || apellido1 || ' ' || apellido2 AS nombre_completo,
   puesto_id
FROM
   empleados
WHERE
   puesto_id = (SELECT puesto_id
                FROM empleados
                WHERE apellido1 = 'Valdes')
   AND apellido1 <> 'Valdes'; -- Excluimos al propio Valdes
```

Errores comunes:

- Si la subconsulta no devuelve ninguna fila, la condición de la consulta externa da `NULL` (no devuelve datos, pero no falla).
- Si la subconsulta devuelve **más de una fila** y usamos un operador escalar (como =), Oracle lanzará el error: `ORA-01427: single-row subquery returns more than one row.`

8.2. Subconsultas de Múltiples Filas

Devuelven **más de una fila** a la consulta externa. Deben utilizarse con operadores de conjuntos múltiples:

- `IN`: Igual a cualquiera de los miembros de la lista.
- `ANY`: Compara con cualquier valor devuelto (se usa con =, >, etc.).
- `ALL`: Compara con todos los valores devueltos.

8.2.1. Operador IN

Código 59: Uso de IN en subconsultas

```
-- Buscar empleados que trabajan en departamentos ubicados en
-- la ciudad de 'Albacete' (necesitamos cruzar datos con DEPARTAMENTOS
-- y UBICACIONES, o usar subconsultas).
-- Asumimos que la ubicacion_id de Albacete es 2300.

SELECT nombre, apellido1, apellido2
FROM empleados
WHERE departamento_id IN (SELECT id
                          FROM departamentos
                          WHERE ubicacion_id = 2300);
```

8.2.2. Operadores ANY y ALL

Son menos frecuentes pero muy potentes.

- > `ANY`: Mayor que el mínimo (mayor que cualquiera de la lista).
- < `ANY`: Menor que el máximo.
- > `ALL`: Mayor que el máximo (mayor que todos los de la lista).
- < `LL`: Menor que el mínimo.

Código 60: Uso de ALL

```
-- Empleados que ganan más que TODOS los empleados del departamento 200
-- (Es decir, ganan mas que el que mas gana del depto 200)
SELECT nombre, salario
FROM empleados
WHERE salario > ALL (SELECT salario
                     FROM empleados
                     WHERE departamento_id = 200);
```

Peligro con valores NULL y NOT IN

Se debe tener mucho cuidado al usar `NOT IN` si la subconsulta devuelve algún valor `NULL`.
Si la lista de valores contiene un `NULL`, la operación lógica `NOT IN` siempre evaluará a `UNKNOWN` y la consulta externa **no devolverá ninguna fila**, aunque parezca que debería.

8.3. Subconsultas Correlacionadas

Hasta ahora, las subconsultas se ejecutaban una sola vez al principio y pasaban el resultado a la consulta externa.

En una **subconsulta correlacionada**, la subconsulta utiliza una columna de la consulta externa. Por lo tanto, la subconsulta se ejecuta **una vez por cada fila candidata** de la tabla externa.

SINTAXIS GENÉRICA

```
SELECT outer.columna
FROM tabla outer
WHERE columna operador (SELECT columna
                        FROM tabla inner
                        WHERE inner.columna = outer.columna);
```

Este tipo de consultas son fundamentales cuando queremos comparar una fila con un agregado de su propio grupo.

Código 61: Subconsulta Correlacionada

```
-- Empleados que ganan más que el salario promedio DE SU PROPIO
   departamento.
-- Para cada empleado, calculamos el promedio de su departamento y
   comparamos.

SELECT
   e1.nombre,
   e1.departamento_id,
   e1.salario
FROM
   empleados e1
WHERE
   salario > (SELECT AVG(e2.salario)
              FROM empleados e2
              WHERE e2.departamento_id = e1.departamento_id);
```

8.3.1. Operador EXISTS

El operador `EXISTS` verifica si la subconsulta devuelve al menos una fila. Es muy eficiente porque detiene la búsqueda en la subconsulta tan pronto encuentra la primera coincidencia.

Código 62: Uso de EXISTS

```
-- Buscar departamentos que tengan al menos un empleado.
-- (Alternativa eficiente al IN)
SELECT
   nombre
FROM
   departamentos d
WHERE EXISTS (SELECT 1
              FROM empleados e
              WHERE e.departamento_id = d.id);
```

9. Introducción a Funciones Analíticas

Las funciones analíticas (o *Window Functions*) son una de las características más potentes de Oracle. Permiten realizar cálculos sobre un conjunto de filas relacionadas con la fila actual, sin necesidad de agrupar el resultado en una sola fila (como hace el `GROUP BY`).

Mientras que `GROUP BY` reduce el número de filas devueltas, las funciones analíticas mantienen todas las filas y añaden una columna con el resultado del cálculo.

SINTAXIS GENÉRICA

```
FUNCION_ANALITICA() OVER (
    [PARTITION BY columna_agrupacion]
    [ORDER BY columna_ordenacion]
    [ROWS ...]
)
```

9.1. Ranking de Filas (ROW_NUMBER, RANK, DENSE_RANK)

Supongamos que queremos hacer un ranking de los empleados que más ganan, pero reiniciando el ranking para cada departamento.

- `ROW_NUMBER()`: Asigna un número secuencial único (1, 2, 3...). Si hay empates, decide arbitrariamente quién va antes.
- `RANK()`: Asigna ranking con saltos si hay empates (ej: 1, 2, 2, 4...).
- `DENSE_RANK()`: Asigna ranking sin saltos si hay empates (ej: 1, 2, 2, 3...).

Código 63: Ranking de salarios por departamento

```
-- Queremos un listado de todos los empleados, mostrando su posicion
-- en el ranking salarial DENTRO de su departamento.

SELECT
   departamento_id,
   nombre,
   salario,
   RANK() OVER (PARTITION BY departamento_id
                ORDER BY salario DESC) AS ranking
FROM
   empleados;
```

El resultado mostrará a todos los empleados. Para cada cambio de `departamento_id`, el ranking volverá a empezar en 1.

9.2. Totales Acumulados

Podemos usar funciones de agregación como `SUM` o `AVG` en modo analítico para calcular, por ejemplo, sumas acumulativas.

Código 64: Suma acumulada de salarios

```
-- Ver como crece el gasto en nomina a medida que contratamos gente.
-- Ordenamos por fecha y vamos sumando el salario al acumulado anterior.

SELECT
   nombre,
   fecha_contratacion,
   salario,
   SUM(salario) OVER (ORDER BY fecha_contratacion) AS gasto_acumulado
FROM
   empleados
WHERE
   departamento_id = 500;
```

9.3. Comparación con filas previas (LAG y LEAD)

Permiten acceder al valor de una fila anterior (`LAG`) o posterior (`LEAD`) sin necesidad de hacer complejos *Self Joins*.

Código 65: Comparación con el anterior

```
-- Mostrar el salario del empleado y el salario del empleado
-- contratado justo antes que el.

SELECT
   nombre,
   salario,
   LAG(salario, 1) OVER (ORDER BY fecha_contratacion) AS salario_previo
FROM
   empleados;
```